TECHNICAL UNIVERSITY OF MOLDOVA
Faculty of Computers, Informatics, and Microelectronics
Department of Software Engineering
Software Engineering Study Programme

LABORATORY REPORT NO. 4

DES Key Permutation and Subkey Initialization

Cryptography: Task 2.1, student number 12

Student: Gafenco Victor

Group: FAF-241

Coordinator: asist. univ. Zaica Maia

Chișinău - 2026

CONTENTS

Abstract **3**

Introduction **4**

1 Short theory **5**

1.1 Structure of the DES algorithm 5

1.2 Key encoding and Permuted Choice 1 (PC-1) 5

2 Implementation **7**

2.1 ASCII encoding and key validation 7

2.2 PC-1 permutation and intermediate trace 7

2.3 Console interface and build integration 8

3 Results and verification **9**

3.1 Worked example for an 8-symbol key 9

3.2 Console evidence 9

3.3 Automated verification 11

Conclusions **14**

ABSTRACT

This laboratory examines the key initialization stage of the Data Encryption Standard (DES) block cipher and implements Task 2.1 in Java. For student number $n = 12$, the task index is $12 \bmod 11 = 1$, which requires deriving the 56-bit permuted key K^+ from an 8-symbol key. Each printable American Standard Code for Information Interchange (ASCII) character is encoded as an 8-bit binary byte to form the 64-bit key K. Permuted Choice 1 (PC-1) then discards the eight byte-parity bits at positions $8, 16, 24, 32, 40, 48, 56$ and 64, and rearranges the remaining 56 bits into the two 28-bit registers C_0 and D_0 that compose K^+. The console application supports both manual entry and random key generation, prints all tables used and every intermediate bit selection, and is verified by automated JUnit tests.

Keywords: Data Encryption Standard, block cipher, key schedule, Permuted Choice 1, ASCII encoding, Java

INTRODUCTION

The objective of this laboratory work is to study the structure of the Data Encryption Standard (DES) symmetric block cipher and implement Task 2.1 for student order number $n = 12$ (n mod $11 = 12$ mod $11 = 1$). Task 2.1 requires determining the 56-bit permuted key K^+ from an 8-symbol DES key.

The application allows the user either to enter an 8-symbol ASCII key manually or to generate one randomly, validates the input, and displays on the screen all tables used together with every intermediate step of the transformation [1].

1 SHORT THEORY

The Data Encryption Standard (DES) is a symmetric-key Feistel block cipher that encrypts 64-bit plaintext blocks under the control of a 64-bit key block, of which 56 bits participate directly in the cryptographic transformations.

1.1 Structure of the DES algorithm

Encryption of a 64-bit plaintext block M proceeds through three main stages:

- an Initial Permutation (IP) rearranges the 64 bits of M and splits the result into a 32-bit left half L_0 and a 32-bit right half R_0;
- sixteen identical Feistel rounds ($i = 1, \ldots, 16$) update the halves according to:

$$L_i = R_{i-1}, \qquad R_i = L_{i-1} \oplus f(R_{i-1}, K_i), \tag{1.1}$$

  where $\oplus$ denotes bitwise exclusive OR (XOR) and K_i is the 48-bit round subkey;
- after round 16, the swapped 64-bit pre-output $R_{16}L_{16}$ is passed through the final permutation IP^{-1} to produce the 64-bit ciphertext block C;

Inside each round i, the Feistel function $f(R_{i-1}, K_i)$ expands the 32-bit half R_{i-1} to 48 bits via the expansion table E, computes $E(R_{i-1}) \oplus K_i = B_1B_2B_3B_4B_5B_6B_7B_8$ as eight 6-bit blocks, compresses each 6-bit block B_j to 4 bits through substitution box S_j (using the outer two bits for the row index $0 \ldots 3$ and the inner four bits for the column index $0 \ldots 15$), and permutes the concatenated 32-bit output $S_1(B_1) \cdots S_8(B_8)$ through the straight permutation table P. Decryption uses the identical circuit with the round subkeys applied in reverse order (K_{16} down to K_1).

1.2 Key encoding and Permuted Choice 1 (PC-1)

The DES key schedule derives the sixteen 48-bit round keys $K_1, \ldots, K_{16}$ from an initial 8-symbol key $K = s_1s_2s_3s_4s_5s_6s_7s_8$. Each symbol s_j ($j = 1, \ldots, 8$) is converted via its ASCII code into an 8-bit binary byte, ordered from most significant bit (MSB) to least significant bit (LSB). Concatenating the eight bytes yields the 64-bit key block:

$$K = k_1k_2k_3 \ldots k_{64}, \tag{1.2}$$

where byte j occupies bit positions $8(j-1)+1$ through $8j$. The eighth bit of every byte ($k_8, k_{16}, k_{24}, k_{32}, k_{40}, k_{48}, k_{56}$) serves as a parity position and is ignored by the key schedule.

The first step of the key schedule applies Permuted Choice 1 (PC-1), given in Table 1.1, to select

and reorder the 56 non-parity bits of K:

$$K^{+} = k_{\text{PC-1}(1)}\, k_{\text{PC-1}(2)}\, \ldots\, k_{\text{PC-1}(56)} = C_0 D_0. \tag{1.3}$$

Table 1.1 – Permuted Choice 1 (PC-1) bit selection table

Half	Source bit positions in the 64-bit key K						
C_0 (bits 1–7)	57	49	41	33	25	17	9
C_0 (bits 8–14)	1	58	50	42	34	26	18
C_0 (bits 15–21)	10	2	59	51	43	35	27
C_0 (bits 22–28)	19	11	3	60	52	44	36
D_0 (bits 29–35)	63	55	47	39	31	23	15
D_0 (bits 36–42)	7	62	54	46	38	30	22
D_0 (bits 43–49)	14	6	61	53	45	37	29
D_0 (bits 50–56)	21	13	5	28	20	12	4

The first four rows of Table 1.1 form the 28-bit left register C_0 by reading bit columns 1, 2 and 3 of the eight key bytes from byte 8 up to byte 1, followed by column 4 of bytes 8 through 5. The last four rows form the 28-bit right register D_0 by reading columns 7, 6 and 5 from byte 8 up to byte 1, followed by column 4 of bytes 4 through 1. In subsequent stages of the DES key schedule, C_{i-1} and D_{i-1} undergo circular left shifts of one bit (in rounds $1, 2, 9, 16$) or two bits (in all other rounds), and Permuted Choice 2 (PC-2) selects 48 bits from $C_i D_i$ to form each round key K_i.

2 IMPLEMENTATION

The Java implementation is organized in the `md.utm.cryptography.lab4` package and separates the cryptographic key transformation from the interactive console interface [1].

2.1 ASCII encoding and key validation

`DesKeyPermutation` validates 8-symbol keys, supports random generation over printable alphanumeric ASCII characters, and converts each symbol into an 8-bit binary string. Validation inspects every Unicode code point to reject control characters and non-ASCII symbols outside the printable range `U+0020`–`U+007E`, and verifies that the key length is exactly 8 characters:

```
public static String toBinaryKey(String key) {
    String validated = validateKey(key);
    StringBuilder bits = new StringBuilder(64);
    for (int i = 0; i < validated.length(); i++) {
        bits.append(toEightBitBinary(validated.charAt(i)));
    }
    return bits.toString();
}
```

Each character code is formatted with `Integer.toBinaryString` and left-padded with zeros to 8 bits, preserving the bit numbering from k_1 (MSB of byte 1) to k_{64} (LSB of byte 8).

2.2 PC-1 permutation and intermediate trace

The constant array `PC1` stores the 56 one-based bit positions from Table 1.1. The method `applyPc1` verifies that the input string contains 64 binary digits and maps each one-based position `PC1[j]` to zero-based string index `PC1[j] - 1`:

```
public static String applyPc1(String binaryKey) {
    if (binaryKey == null || binaryKey.length() != 64
            || !binaryKey.chars().allMatch(ch -> ch == '0' || ch == '1')) {
        throw new IllegalArgumentException(
                "Binary key K must contain exactly 64 bits ('0' or '1').");
    }
    StringBuilder permuted = new StringBuilder(PC1.length);
    for (int position : PC1) {
        permuted.append(binaryKey.charAt(position - 1));
    }
    return permuted.toString();
}
```

The methods `c0()` and `d0()` return the 28-bit halves `kPlus.substring(0, 28)` and `kPlus.substring(28, 56)`. The method `formatTrace()` builds the complete console report: the byte-by-byte ASCII and binary breakdown table (highlighting the 7 data bits and the 8th parity bit of each byte), the 64-bit key K in hexadecimal and 8-bit blocks, the 8-row PC-1 table paired with every intermediate bit lookup $k_{\text{PC-1}(j)}$, and the resulting registers C_0, D_0 and K^+.

2.3 Console interface and build integration

`Main` provides an interactive loop with three options: manual 8-symbol key entry (`1`), random 8-symbol key generation (`2`), and exit (`0`). Random keys are generated with `SecureRandom` during interactive use, while an optional `-Dlab4.seed` system property enables deterministic regeneration of console transcripts. Invalid menu selections or malformed keys display an explanation and retry the current prompt without terminating the session. The `lab4` Maven module compiles under the root aggregator and packages an executable JAR with `md.utm.cryptography.lab4.Main` as its entry point.

3 RESULTS AND VERIFICATION

The implementation is verified through a step-by-step worked example, captured console sessions for manual, random and invalid inputs, and automated unit tests.

3.1 Worked example for an 8-symbol key

For the 8-symbol ASCII key `CRYPTO26`, Table 3.1 gives the byte-level encoding into the 64-bit key K.

Table 3.1 – ASCII and binary representation of the 8-symbol key `CRYPTO26`

Byte	Symbol	ASCII (dec / hex)	Bit range	8-bit binary	Data bits (1–7)	Parity bit
1	C	67 / 0x43	1–8	01000011	0100001	$k_8 = 1$
2	R	82 / 0x52	9–16	01010010	0101001	$k_{16} = 0$
3	Y	89 / 0x59	17–24	01011001	0101100	$k_{24} = 1$
4	P	80 / 0x50	25–32	01010000	0101000	$k_{32} = 0$
5	T	84 / 0x54	33–40	01010100	0101010	$k_{40} = 0$
6	O	79 / 0x4F	41–48	01001111	0100111	$k_{48} = 1$
7	2	50 / 0x32	49–56	00110010	0011001	$k_{56} = 0$
8	6	54 / 0x36	57–64	00110110	0011011	$k_{64} = 0$

Because every standard 7-bit ASCII code lies below 128, the most significant bit of each byte ($k_1, k_9, k_{17}, k_{25}, k_{33}, k_{41}, k_{49}, k_{57}$) equals `0`. Applying Table 1.1 extracts those eight bits first into C_0, yielding:

$$C_0 = \texttt{0000000 0001111 1111000 0001101}, \tag{3.1}$$

$$D_0 = \texttt{1110001 1101100 0000100 1001110}, \tag{3.2}$$

$$K^+ = \texttt{0000000 0001111 1111000 0001101 1110001 1101100 0000100 1001110}. \tag{3.3}$$

3.2 Console evidence

Figures 3.1, 3.2 and 3.3 show executable-JAR console sessions captured through a pseudo-terminal. They can be regenerated after building with:

```
python3 scripts/capture-results.py --lab lab4
```

Figure 3.1 demonstrates manual input of the 8-symbol key `CRYPTO26`, displaying the ASCII encoding table, the PC-1 table, the intermediate bit selections, and the resulting 56-bit key K^+.

```
Laboratory 4 — DES key permutation K+ (task 2.1, student 12)
Mode (1 = manual 8-symbol key, 2 = random 8-symbol key, 0 = exit): 1
Key (exactly 8 printable ASCII symbols): CRYPTO26
Step 1 — ASCII encoding and 64-bit key K (bits 8, 16, ..., 64 are parity bits):
Byte  Symbol  ASCII (dec/hex)  Bits    8-bit binary  Data (1..7)  Parity (8)
1     'C'       67 / 0x43   1..8    01000011      0100001      1 (k8)
2     'R'       82 / 0x52   9..16   01010010      0101001      0 (k16)
3     'Y'       89 / 0x59   17..24  01011001      0101100      1 (k24)
4     'P'       80 / 0x50   25..32  01010000      0101000      0 (k32)
5     'T'       84 / 0x54   33..40  01010100      0101010      0 (k40)
6     'O'       79 / 0x4F   41..48  01001111      0100111      1 (k48)
7     '2'       50 / 0x32   49..56  00110010      0011001      0 (k56)
8     '6'       54 / 0x36   57..64  00110110      0011011      0 (k64)
K (hex): 43 52 59 50 54 4F 32 36
K (64b): 01000011 01010010 01011001 01010000 01010100 01001111 00110010 00110110

Step 2 — Permuted Choice 1 (PC-1) table and intermediate bit selection:
Half  Row  PC-1 bit positions    Selected bits from K                       Row bits
C0      1  57 49 41 33 25 17  9  k57=0 k49=0 k41=0 k33=0 k25=0 k17=0 k9=0   0000000
C0      2   1 58 50 42 34 26 18  k1=0  k58=0 k50=0 k42=1 k34=1 k26=1 k18=1  0001111
C0      3  10  2 59 51 43 35 27  k10=1 k2=1  k59=1 k51=1 k43=0 k35=0 k27=0  1111000
C0      4  19 11  3 60 52 44 36  k19=0 k11=0 k3=0  k60=1 k52=1 k44=0 k36=1  0001101
D0      5  63 55 47 39 31 23 15  k63=1 k55=1 k47=1 k39=0 k31=0 k23=0 k15=1  1110001
D0      6   7 62 54 46 38 30 22  k7=1  k62=1 k54=0 k46=1 k38=1 k30=0 k22=0  1101100
D0      7  14  6 61 53 45 37 29  k14=0 k6=0  k61=0 k53=0 k45=1 k37=0 k29=0  0000100
D0      8  21 13  5 28 20 12  4  k21=1 k13=0 k5=0  k28=1 k20=1 k12=1 k4=0   1001110

Step 3 — Output 56-bit permuted key K+:
C0 (28 bits): 0000000 0001111 1111000 0001101
D0 (28 bits): 1110001 1101100 0000100 1001110
K+ (56 bits): 0000000 0001111 1111000 0001101 1110001 1101100 0000100 1001110

Mode (1 = manual 8-symbol key, 2 = random 8-symbol key, 0 = exit): 0
```

Figure 3.1 – Manual 8-symbol key entry and intermediate PC-1 trace

Figure 3.2 shows mode `2`, where the program generates the random 8-symbol key `Q4G5wDqb` and computes $K^+ =$ `0000000 0111101 0111011 0100101 1001010 0001111 1000000 0001011`.

```
Laboratory 4 — DES key permutation K+ (task 2.1, student 12)
Mode (1 = manual 8-symbol key, 2 = random 8-symbol key, 0 = exit): 2
Generated random key: Q4G5wDqb
Step 1 — ASCII encoding and 64-bit key K (bits 8, 16, ..., 64 are parity bits):
Byte  Symbol  ASCII (dec/hex)  Bits    8-bit binary  Data (1..7)  Parity (8)
1     'Q'       81 / 0x51   1..8    01010001      0101000      1 (k8)
2     '4'       52 / 0x34   9..16   00110100      0011010      0 (k16)
3     'G'       71 / 0x47   17..24  01000111      0100011      1 (k24)
4     '5'       53 / 0x35   25..32  00110101      0011010      1 (k32)
5     'w'      119 / 0x77   33..40  01110111      0111011      1 (k40)
6     'D'       68 / 0x44   41..48  01000100      0100010      0 (k48)
7     'q'      113 / 0x71   49..56  01110001      0111000      1 (k56)
8     'b'       98 / 0x62   57..64  01100010      0110001      0 (k64)
K (hex): 51 34 47 35 77 44 71 62
K (64b): 01010001 00110100 01000111 00110101 01110111 01000100 01110001 01100010

Step 2 — Permuted Choice 1 (PC-1) table and intermediate bit selection:
Half  Row  PC-1 bit positions    Selected bits from K                      Row bits
C0      1  57 49 41 33 25 17  9  k57=0 k49=0 k41=0 k33=0 k25=0 k17=0 k9=0   0000000
C0      2   1 58 50 42 34 26 18  k1=0  k58=1 k50=1 k42=1 k34=1 k26=0 k18=1  0111101
C0      3  10  2 59 51 43 35 27  k10=0 k2=1  k59=1 k51=1 k43=0 k35=1 k27=1  0111011
C0      4  19 11  3 60 52 44 36  k19=0 k11=1 k3=0  k60=0 k52=1 k44=0 k36=1  0100101
D0      5  63 55 47 39 31 23 15  k63=1 k55=0 k47=0 k39=1 k31=0 k23=1 k15=0  1001010
D0      6   7 62 54 46 38 30 22  k7=0  k62=0 k54=0 k46=1 k38=1 k30=1 k22=1  0001111
D0      7  14  6 61 53 45 37 29  k14=1 k6=0  k61=0 k53=0 k45=0 k37=0 k29=0  1000000
D0      8  21 13  5 28 20 12  4  k21=0 k13=0 k5=0  k28=1 k20=0 k12=1 k4=1   0001011

Step 3 — Output 56-bit permuted key K+:
C0 (28 bits): 0000000 0111101 0111011 0100101
D0 (28 bits): 1001010 0001111 1000000 0001011
K+ (56 bits): 0000000 0111101 0111011 0100101 1001010 0001111 1000000 0001011

Mode (1 = manual 8-symbol key, 2 = random 8-symbol key, 0 = exit): 0
```

Figure 3.2 – Random 8-symbol key generation and derivation of K^+

Figure 3.3 demonstrates input validation: the program rejects menu option `3`, a 5-character key (`SHORT`), a 10-character key (`TOOLONGKEY`), and a key containing a non-ASCII character (`Română12`), before accepting `DESKEY01`.

3.3 Automated verification

Five JUnit tests in `DesKeyPermutationTest` verify the implementation:

- comparison against the standard 64-bit hexadecimal test vector `133457799BBCDFF1`, confirming that `applyPc1` produces $C_0 =$ `1111000 0110011 0010101 0101111` and $D_0 =$ `0101010 101`
- verification of the complete ASCII-to-K^+ pipeline and formatted trace for `CRYPTO26`;
- exhaustive structural testing across all 56 unit-bit vectors, proving that PC-1 is a bijection from $\{1, \ldots, 64\} \setminus \{8, 16, 24, 32, 40, 48, 56, 64\}$ onto $\{1, \ldots, 56\}$ and that setting all eight parity bits to `1` yields an all-zero K^+;

```
Laboratory 4 — DES key permutation K+ (task 2.1, student 12)
Mode (1 = manual 8-symbol key, 2 = random 8-symbol key, 0 = exit): 3
Invalid input: Choose an integer from 0 to 2.
Mode (1 = manual 8-symbol key, 2 = random 8-symbol key, 0 = exit): 1
Key (exactly 8 printable ASCII symbols): SHORT
Invalid input: Key must contain exactly 8 printable ASCII symbols (got 5).
Key (exactly 8 printable ASCII symbols): TOOLONGKEY
Invalid input: Key must contain exactly 8 printable ASCII symbols (got 10).
Key (exactly 8 printable ASCII symbols): Română12
Invalid input: Rejected character 'â' (U+00E2). Allowed symbols: printable ASCII (U+0020 to U+007E).
Key (exactly 8 printable ASCII symbols): DESKEY01
Step 1 — ASCII encoding and 64-bit key K (bits 8, 16, ..., 64 are parity bits):
Byte  Symbol  ASCII (dec/hex)  Bits    8-bit binary  Data (1..7)  Parity (8)
1     'D'       68 / 0x44   1..8    01000100      0100010      0 (k8)
2     'E'       69 / 0x45   9..16   01000101      0100010      1 (k16)
3     'S'       83 / 0x53   17..24  01010011      0101001      1 (k24)
4     'K'       75 / 0x4B   25..32  01001011      0100101      1 (k32)
5     'E'       69 / 0x45   33..40  01000101      0100010      1 (k40)
6     'Y'       89 / 0x59   41..48  01011001      0101100      1 (k48)
7     '0'       48 / 0x30   49..56  00110000      0011000      0 (k56)
8     '1'       49 / 0x31   57..64  00110001      0011000      1 (k64)
K (hex): 44 45 53 4B 45 59 30 31
K (64b): 01000100 01000101 01010011 01001011 01000101 01011001 00110000 00110001

Step 2 — Permuted Choice 1 (PC-1) table and intermediate bit selection:
Half  Row  PC-1 bit positions    Selected bits from K                       Row bits
C0      1  57 49 41 33 25 17  9  k57=0 k49=0 k41=0 k33=0 k25=0 k17=0 k9=0   0000000
C0      2   1 58 50 42 34 26 18  k1=0  k58=0 k50=0 k42=1 k34=1 k26=1 k18=1  0001111
C0      3  10  2 59 51 43 35 27  k10=1 k2=1  k59=1 k51=1 k43=0 k35=0 k27=0  1111000
C0      4  19 11  3 60 52 44 36  k19=0 k11=0 k3=0  k60=1 k52=1 k44=1 k36=0  0001110
D0      5  63 55 47 39 31 23 15  k63=0 k55=0 k47=0 k39=0 k31=1 k23=1 k15=0  0000110
D0      6   7 62 54 46 38 30 22  k7=0  k62=0 k54=0 k46=0 k38=1 k30=0 k22=0  0000100
D0      7  14  6 61 53 45 37 29  k14=1 k6=1  k61=0 k53=0 k45=1 k37=0 k29=1  1100101
D0      8  21 13  5 28 20 12  4  k21=0 k13=0 k5=0  k28=0 k20=1 k12=0 k4=0   0000100

Step 3 — Output 56-bit permuted key K+:
C0 (28 bits): 0000000 0001111 1111000 0001110
D0 (28 bits): 0000110 0000100 1100101 0000100
K+ (56 bits): 0000000 0001111 1111000 0001110 0000110 0000100 1100101 0000100

Mode (1 = manual 8-symbol key, 2 = random 8-symbol key, 0 = exit): 0
```

Figure 3.3 – Validation of menu choices, key length and non-ASCII characters

- verification of 25 randomly generated 8-symbol keys;
- rejection of `null`, wrong-length keys, non-ASCII characters, control characters, and malformed 64-bit binary strings;

CONCLUSIONS

Task 2.1 of Laboratory Work No. 4 is implemented in Java, converting any 8-symbol printable ASCII key into its 64-bit binary representation K and applying Permuted Choice 1 (PC-1) to obtain the 56-bit permuted key $K^+ = C_0D_0$. Displaying the byte-level ASCII table alongside the row-by-row PC-1 bit selections makes both parity-bit removal and cross-byte bit transposition explicit.

The experiment also highlights an important practical property of ASCII-entered DES keys: because standard 7-bit ASCII characters always have a zero most significant bit in every byte ($k_1, k_9, \ldots, k_{57} = 0$) while the eighth bit ($k_8, k_{16}, \ldots, k_{64}$) is discarded by PC-1, an 8-character printable ASCII key supplies at most $8 \times 6 = 48$ variable bits out of the 56 bits in K^+, with the first eight bits of C_0 fixed to zero.

BIBLIOGRAPHY

[1] Gafenco Victor. *Cryptography Laboratory Repository* [online]. GitHub, 2026 [accessed 09.11.2026]. Available: `https://github.com/yoda-dinmd/crypt-labs`